



Bases de Datos y Recursos de Información – 2026

Trabajo Práctico: MovieWeb

Fecha de entrega: domingo, 5 de noviembre de 2026, 23:59 hs.

Fechas de exposición: lunes 9 de noviembre y jueves 12 de noviembre de 2026.

Grupos: 6 grupos de cuatro (4) personas.

Objetivos

El objetivo principal de este trabajo práctico es diseñar e implementar una aplicación web completa que interactúa con bases de datos relacionales y no relacionales. La aplicación, llamada **MovieWeb**, permitirá a los usuarios buscar y explorar información sobre películas, actores y directores.

Los objetivos específicos son:

- Adquirir experiencia en el modelado y la manipulación de una base de datos relacional **PostgreSQL**.
 - Introducir el uso de una base de datos NoSQL **MongoDB** para registrar actividades de usuario y gestionar contenido flexible.
 - Integrar la aplicación con una API externa (TMDB) para enriquecer la experiencia de usuario con contenido multimedia.
 - Desarrollar buenas prácticas de configuración de un entorno de desarrollo Node.js, incluyendo el manejo de variables de entorno y scripts de automatización.
-

Software y recursos

Software requerido:

1. **Node.js y npm** (Node Package Manager): para ejecutar la aplicación web.
2. **PostgreSQL**: para gestionar la base de datos relacional "movies".
3. **MongoDB**: para la base de datos NoSQL que registrará la actividad de los usuarios.

4. Un editor de código (Visual Studio Code, WebStorm, etc.).

Código provisto: se proporciona un código base (scaffolding) que servirá como punto de partida. Este incluye la estructura de la aplicación Node.js, rutas, vistas y la base de datos inicial "movies" para PostgreSQL.

Recursos externos: para mejorar el diseño y la apariencia del sitio, deberán obtener una **clave de API de The Movie Database (TMDB)**. Esto les permitirá incluir carátulas de películas, fotos de actores, trailers, enlaces y otra información relevante.

Instrucciones

1. Configuración del entorno y preparación

- **Instalación:** instalar Node.js, PostgreSQL y MongoDB.
- **Base de datos:** configurar PostgreSQL y crear la base de datos "movies" a partir del script proporcionado.
- **Código:** descargar el código base.
- **Dependencias:** instalar las dependencias del proyecto con el comando `npm install`.
- **Variables de entorno:**
 - Crear un archivo `.env` a partir del modelo proporcionado `.env_ejemplo`.
 - Completar el archivo `.env` con las credenciales de conexión a la base de datos y la API Key de TMDB.
- **Automatización:**
 - Crear un mecanismo automático de configuración del entorno (ej. `npm run dev:setup`). Este script deberá ser capaz de inicializar la base de datos, crear las tablas adicionales requeridas y, si es necesario, insertar datos de prueba.
 - Incluir los scripts SQL para la creación de tablas adicionales y también scripts para el borrado de las mismas, que faciliten la corrección del trabajo.

2. Desarrollo de funcionalidades (PostgreSQL - Base Relacional)

2.1. Búsqueda general (películas, actores y directores)

- Modificar la página principal (`index.ejs`) y la lógica del servidor (`app.js`) para que el buscador principal permita buscar simultáneamente películas, actores y directores.
- Crear una página de resultados de búsqueda (`resultado.ejs`) que muestre los resultados de forma clara y organizada en tres secciones separadas: "Películas", "Actores" y "Directores".

2.2. Páginas de perfil de personas (actores y directores)

- Al hacer clic en el nombre de un actor o director en cualquier parte del sitio (ej. página de resultados), se deberá redirigir a una página de perfil dedicada (`actor.ejs` o `director.ejs`).

- Esta página debe mostrar la información de la persona y un listado de todas las películas en las que ha participado, especificando su rol.

2.3. Página de detalle de película

- Mejorar la página de detalle de película (`pelicula.ejs`) para mostrar toda la información disponible en la base de datos PostgreSQL (género, país, año, etc.).
- Enriquecer esta vista utilizando los datos obtenidos de la API de TMDb, como la carátula (póster), sinopsis, videos (tráileres) y enlaces relevantes.

2.4. Búsqueda por palabra clave (keywords)

- Implementar una funcionalidad de búsqueda específica por palabras clave (keywords).
- Crear las páginas necesarias (`search_keyword.ejs` y `resultados_keyword.ejs`) para que un usuario pueda ingresar una palabra clave y ver un listado de todas las películas relacionadas.

2.5. Gestión de usuarios e interacciones

- Diseñar y crear las tablas necesarias en PostgreSQL para la gestión de usuarios (`users`) y sus interacciones con las películas (`user_movies`).
- La estructura debe permitir:
 - Alta, baja y modificación de usuarios (con campos como `user_id`, `username`, `name`, `email`).
 - Asociar películas a los usuarios (ej. como "favoritas" o "vistas").
 - Almacenar una puntuación y una opinión/reseña breve que un usuario asigna a una película.

3. Funcionalidades NoSQL (MongoDB)

3.1. Timeline de actividad del usuario

- Utilizando MongoDB, registrar la actividad relevante de los usuarios en una colección `user_activity`.
- Crear una página de perfil de usuario donde se muestre un "timeline" o feed con su actividad reciente.
- Se deben registrar al menos tres tipos de eventos distintos:
 - "calificó la película X con Y estrellas"
 - "añadió la película X a su lista de favoritos"
 - "escribió una reseña para la película X"
- El esquema de los documentos en MongoDB debe ser flexible para adaptarse a cada tipo de evento.

Ejemplo de documentos en la colección `user_activity` de MongoDB:

// Evento de calificación

```
{
```

```

    "_id": ObjectId("..."),

    "userId": "user_abc",

    "type": "RATED_MOVIE",

    "timestamp": ISODate("2026-10-27T12:00:00Z"),

    "details": { "movieId": 54321, "movieTitle": "El Origen", "rating": 5 }

}

// Evento de reseña

{

    "_id": ObjectId("..."),

    "userId": "user_abc",

    "type": "WROTE_REVIEW",

    "timestamp": ISODate("2026-10-26T18:45:00Z"),

    "details": { "movieId": 67890, "movieTitle": "Parásitos", "reviewId": "review_1122" }

}

// Evento de añadido a favoritos

{

    "_id": ObjectId("..."),

    "userId": "user_abc",

    "type": "ADDED_TO_FAVORITES",

    "timestamp": ISODate("2026-10-25T09:30:00Z"),

    "details": { "movieId": 12345, "movieTitle": "El Señor de los Anillos" }

}

```

3.2. Gestión avanzada de reseñas

- Implementar la funcionalidad completa de **búsqueda, actualización y borrado** de las reseñas de películas, las cuales estarán almacenadas en MongoDB para aprovechar su flexibilidad.

4. Entrega

- Cada grupo debe entregar un único archivo comprimido en formato `.zip` llamado **GrupoX-MovieWeb.zip** (donde X es el número del grupo).
- El archivo `.zip` debe contener:
 1. **Todo el código fuente** de la aplicación.
 2. Un archivo **README.md** en la raíz del proyecto con la siguiente información:
 - Nombres de los integrantes del grupo.
 - Instrucciones completas y claras para la instalación y ejecución del proyecto:
 - Clonado del repositorio (en caso de existir).
 - Instalación de dependencias (`npm install`).
 - Configuración del archivo `.env`.
 - Ejecución del script de configuración automática (`npm run dev:setup`).
 - Comando para iniciar la aplicación (ej. `npm start`).
 3. El archivo de ambiente `.env` con todas las variables de entorno necesarias.
 4. La **clave de API de TMDB (TMDB_API_KEY)** para que el trabajo pueda ser evaluado. Al finalizar la evaluación, podrán revocar dicha clave.

5. Presentación

- Preparar una presentación para demostrar en vivo la aplicación y todas sus funcionalidades implementadas.
- Explicar las decisiones de diseño de las bases de datos (tanto relacional como NoSQL).
- Describir los desafíos técnicos enfrentados durante el desarrollo y cómo fueron resueltos.